

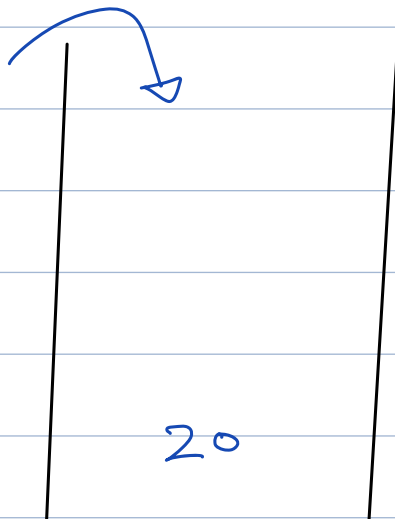
⇒ Stack of plates.

Stack Data Structure

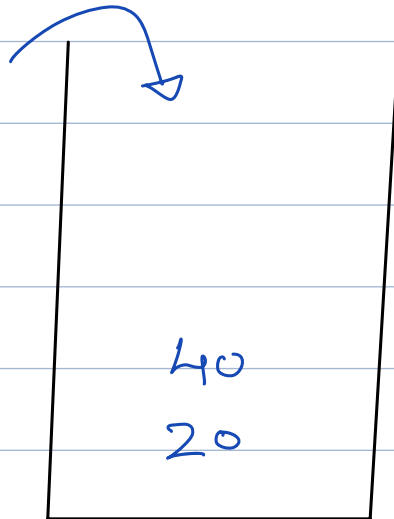


1) Insertion and removal
both happen from the
top

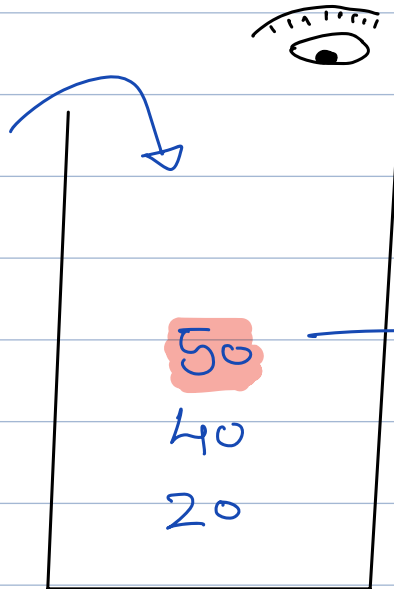
Insert 20



Insert 40

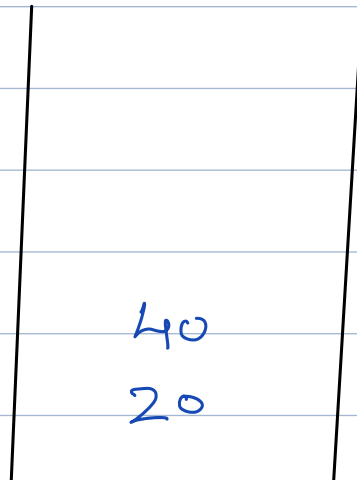


Insert 50



Only element
at the top is
accessible

Delete()



50 → get
removed.

LIFO
LAST IN FIRST OUT

```

int add(int x, int y) {
    return x + y;
}

int product(int x, int y) {
    return x * y;
}

int subtract(int x, int y) {
    return x - y;
}

public static void main() {
    int x = 10; ✓
    int y = 20; ✓
    int temp1 = add(x, y);
    int temp2 = product(x, y);
    int temp3 = subtract(x, y);
    System.out.println(temp1 + temp2 + temp3);
}

```

~~Subtract~~

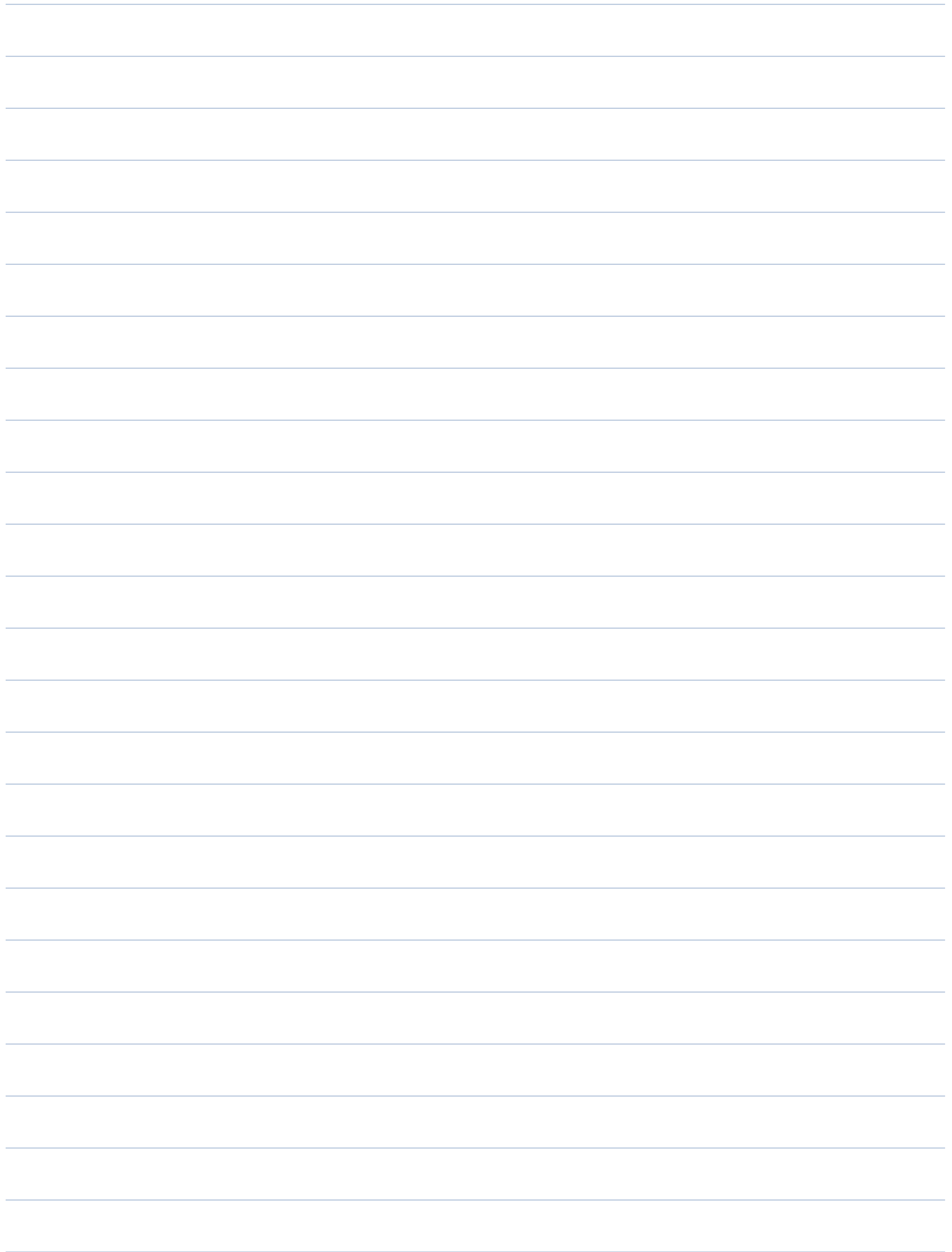
~~product~~

~~add()~~

~~x = 10 y = 20
 temp1 = 30
 temp2 = 200
 temp3 = -10~~

Stack frame. →

Call Stack.



Two Types of Memory

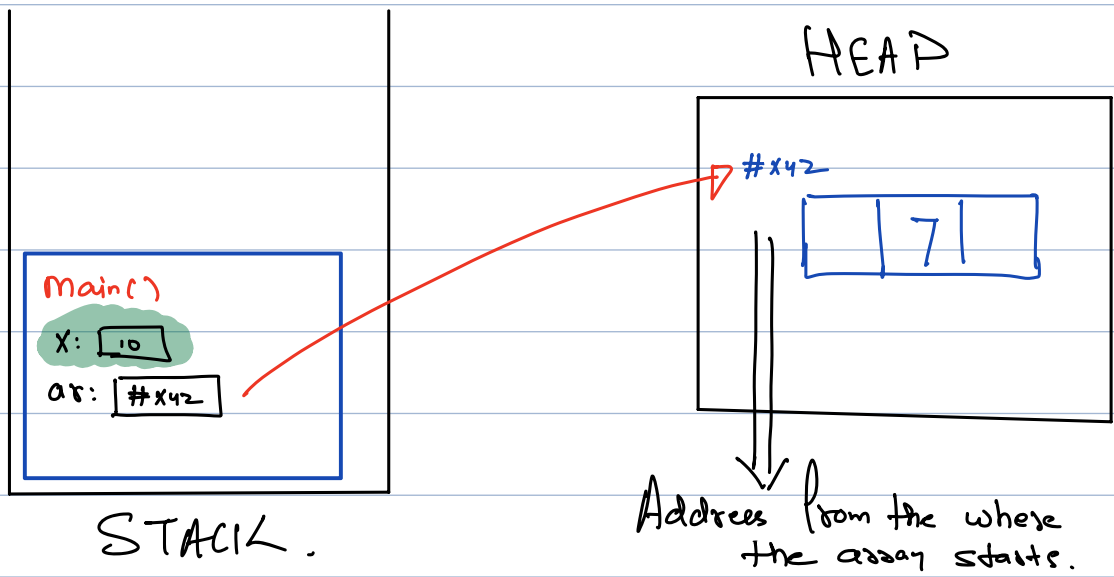
⇒ int, float, double

- 1) Stack: All the primitive data types & reference to non-primitive data types are stored here.
- 2) Heap: Container of the reference is store here.
Eg: Arrays, Objects are created in heap.

Example 1

Consider the below code:

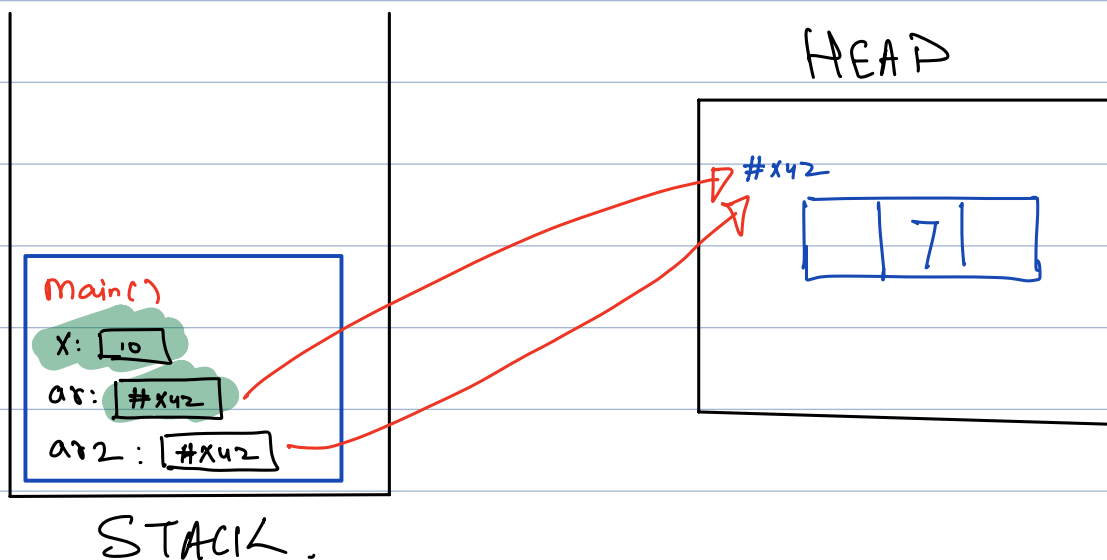
```
public static void main() {  
    int x = 10; ✓  
    int[] ar = new int[3];  
    System.out.println(ar); // #ad1  
    System.out.println(ar[2]); // 0  
    ar[1] = 7;  
}
```



Example 2

Consider the below code:

```
public static void main() {  
    int x = 10;  
    int[] ar = new int[3];  
    int[] ar2 = ar;  
    System.out.println(ar); // 4k  
    System.out.println(ar2); // 4k  
}
```

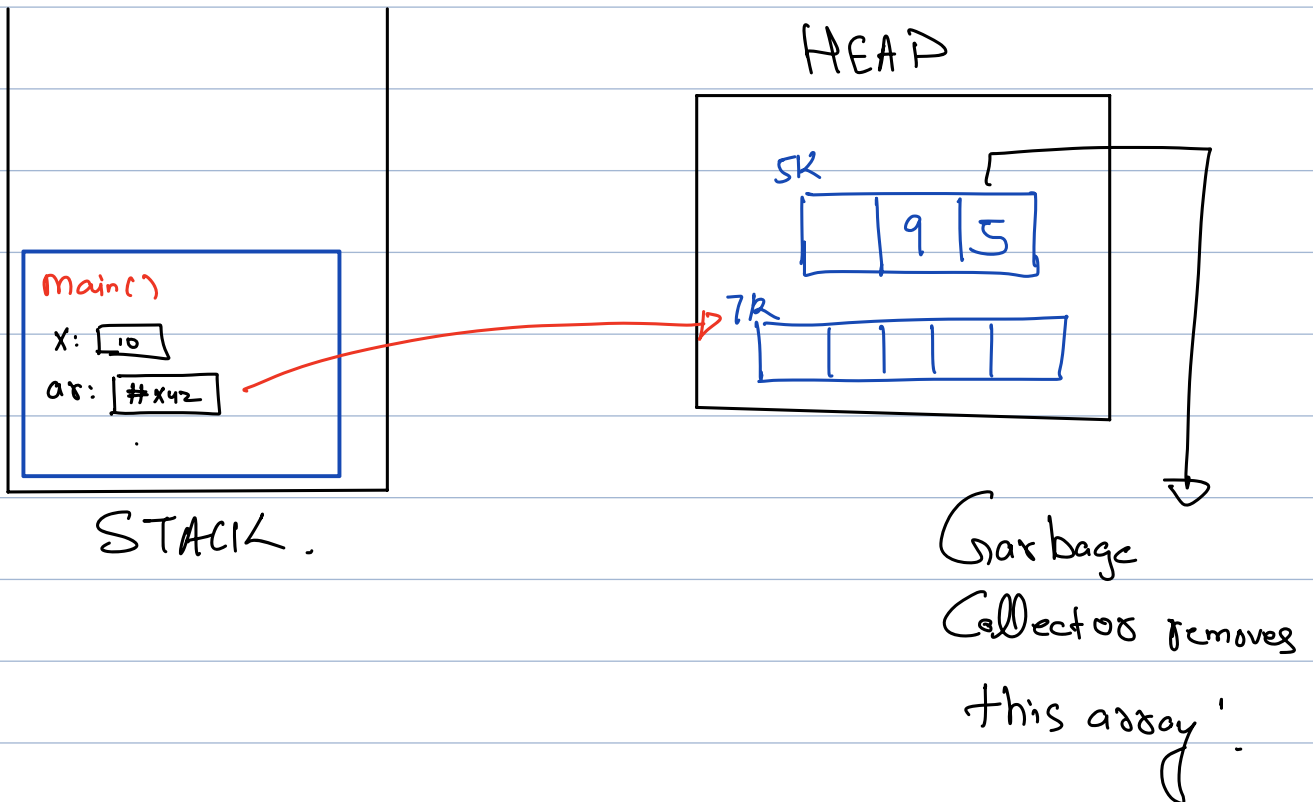


Example 3

Consider the below code:

```
public static void main() {  
    int[] ar = new int[3];  
    System.out.println(ar); // 5k  
    ar[1] = 9;  
    ar[2] = 5;  
  
    ar = new int[5];  
    System.out.println(ar); // 7k  
}
```

points to the new array.



In Java input parameter are passed by value.

1) Primitive Data type

main() {

int x = 10

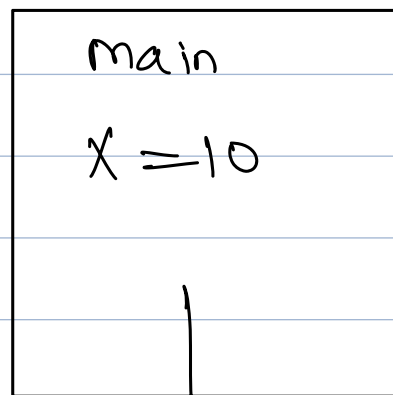
func(x);
print(x); // 10

}

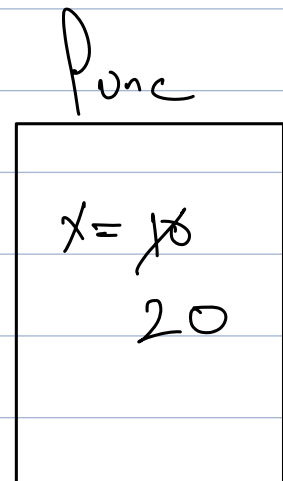
func(int x) {

x = 20

}



10



2) Non primitive data type.

main() {

int arr[100];

arr[1] = 10

func(arr);

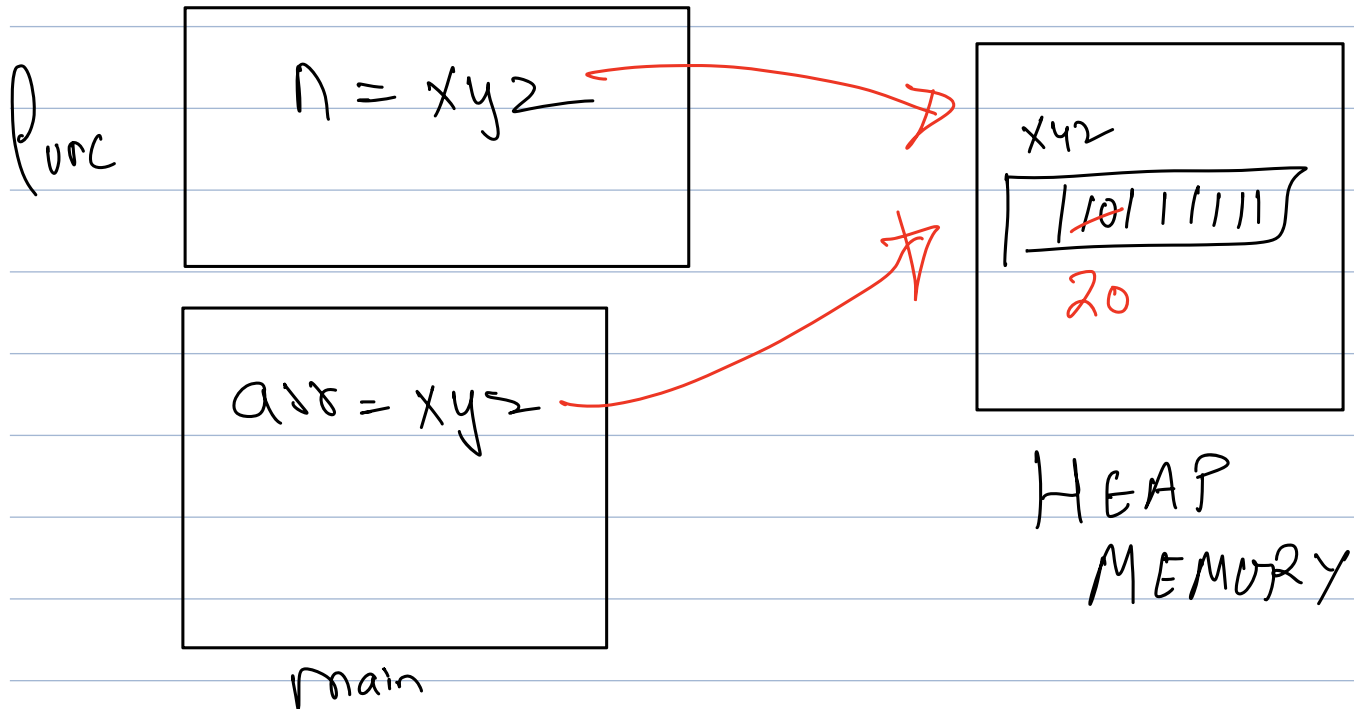
printf(arr[1]); //2

func(int *n) {

*n[1] = 20;

}

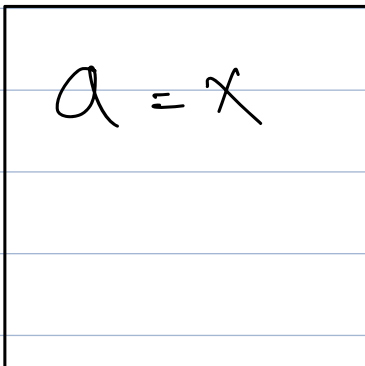
}



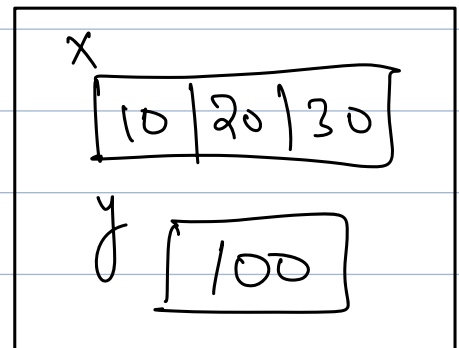
Predict the output:

```
function fun(a):  
    a = new array[1]  
    a[0] = 100  
  
function main():  
    a = [10, 20, 30]  
    fun(a)  
    print(a[0])
```

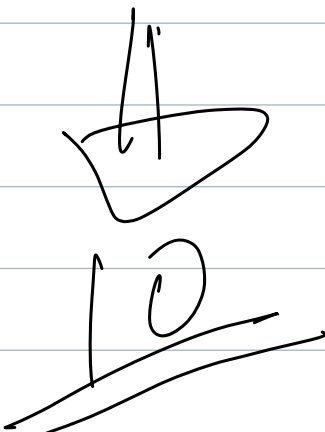
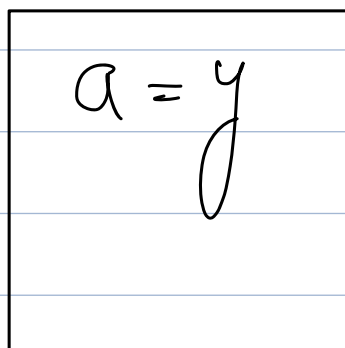
main



HEAP



fun



Sorting: Arranges the elements in
ascending or descending values
based on some factor

→ 95% of cases
factor is value.

Eg

	1	13	9	6	12
	⇓	⇓	⇓	⇓	⇓
no of factors	1	2	3	4	6

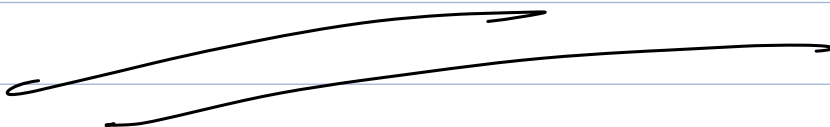
Sorting Libraries.

Java : `Arrays.sort()`
`Collections.sort()`

C++ : `sort()`;

Python : `arr.sort()`

TC: $O(n \log n)$



ELEMENTS REMOVAL

Q Given an arr. of n integers.
You have minimize the total cost
of empty the array. The cost of
removing 1 element is equal to
the sum of all remaining element.

arr [] : [2 8 10]

↓ remove 2 ⇒ 20

[8 10]

↓ remove 8 ⇒ 18

[10]

↓ remove 10 ⇒ 10

[2 8 10]

⇓ arrange in
desc order

[10 8 2]

⇓ ⇓ ⇓
1 2 3

$$10 + 16 + 6 \Rightarrow 32.$$

sort(arr, -1) // in descending.

int total-cost = 0;

for (int i = 0 ; i < n ; i++) {

total-cost = total-cost + (i+1) * arr[i].

}

return total-cost;

Given an array of distinct elements of size n , find the count of **noble integers**.

Note: $\text{arr}[i]$ is **noble** if count of elements smaller than $\text{arr}[i]$ is equal to $\text{arr}[i]$ where $\text{arr}[i]$ is element at index i .

Example 1

$A[] = \{ 1, -5, 3, 5, -10, 4 \}$

Ans = 3

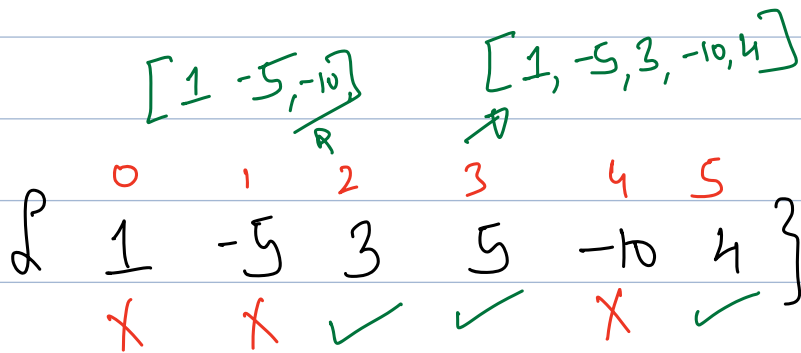
Explanation

For $\text{arr}[2]$ there are three elements less than 3 that is 1, -5 and -10. So $\text{arr}[0]$ is noble integer.

For $\text{arr}[3]$ there are five elements less than 5 that is 1, 3, 4, 5, -5 and -10. So $\text{arr}[3]$ is noble integer.

For $\text{arr}[5]$ there are four elements less than 4 that is 1, 3, -5 and -10. So $\text{arr}[5]$ is noble integer.

In total there are 3 noble elements.



$\Rightarrow 3$

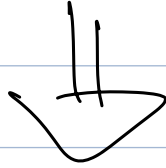
0 1 2 3 4 5
1 -5 3 5 -10 4



0 1 2 3 4 5
-10 -5 1 3 4 5

$\text{arr}[i] = i$

0 1 2 3
[7 3 16 10]



0 1 2 3
[3 7 10 16]



⇒ Sort (arr):

int ans = 0

for (int i = 0; i < n; i++) {
 if (arr[i] == i)
 ans++
}

}

return ans;

WITH DUPLICATES

-10 1 1 3 100
 ✓ ✓ ✓

0	1	2	3	4
-10	1	1	3	100



i is the number of
element smaller than $arr[i]$

↪ FAILS

$arr[i] == arr[i-1]$

0	1	2	3	4	5
-3	1	1	1	5	100

↪

0	1	1	1	4	5
---	---	---	---	---	---

$arr[i] == arr[i-1]$ ↪ FAILS

Count = 0

0	1	2	3	4	5	6	7
2	2	5	6	6	6	8	8

```
sort(arr);    int count = 0  
int ans = 0
```

```
if (arr[0] == 0)  
    ans++;
```

```
for (int i = 1; i < n; i++) {  
    if (arr[i] != arr[i-1]) {  
        count = i  
        if (arr[i] == i) {  
            ans++;  
        }  
    } else {  
        if (arr[i] == count) {  
            ans++;  
        }  
    }  
}
```

TC: $O(n \log n)$

Count = 3

Ans = ~~1~~ 2

0 ✓ 1 ✓ 2 ✓ 3 4
-10 1 1 3 100
A